

CS336: Program Analysis for Security & Privacy
Spring 2026

Privacy Leakage via "Taint Lite"

Field	Details
Project Number	9
Difficulty	Easy
Team Name	Team 9
Date Submitted	April 27, 2026

Team Members

Name	Student ID	Email	Primary Role
Justin Veltri	s1357059	s1357059@monmouth.edu	Project Lead & Taint Engine
Flavia Daniels	s1365459	s1365459@monmouth.edu	Parser & AST Design
Arnav Vasa	s1364337	s1364337@monmouth.edu	Documentation
Garrett Boag	s1353729	s1353729@monmouth.edu	Lexer & Test Suite
Jonathan Veltri	s1357058	s1357058@monmouth.edu	Integration, Reporter & Examples

1. Project Overview

1.1 Problem Statement

Modern software frequently handles Personally Identifiable Information (PII) such as names, email addresses, and Social Security Numbers. A common class of privacy bug occurs when PII is inadvertently written to log files or transmitted to external analytics and API endpoints — often because a developer concatenates a user object into a debug message without realising it contains sensitive fields. These leaks are difficult to catch in code review and may violate GDPR, CCPA, and HIPAA requirements [1][2].

1.2 Approach Summary

Our tool performs a single-pass static taint analysis over programs written in a simplified IMP-Core dialect. It maintains a taint environment — a mapping from variable names to sets of PII tags — and propagates tags through assignments using set union. When a statement passes a tainted value to a known dangerous sink such as `log` or `send_analytics`, the tool reports every PII type reachable at that sink along with a tailored remediation recommendation.

1.3 Language Variant Used

This project uses **IMP-Core (Variant A)** — a minimal imperative language supporting:

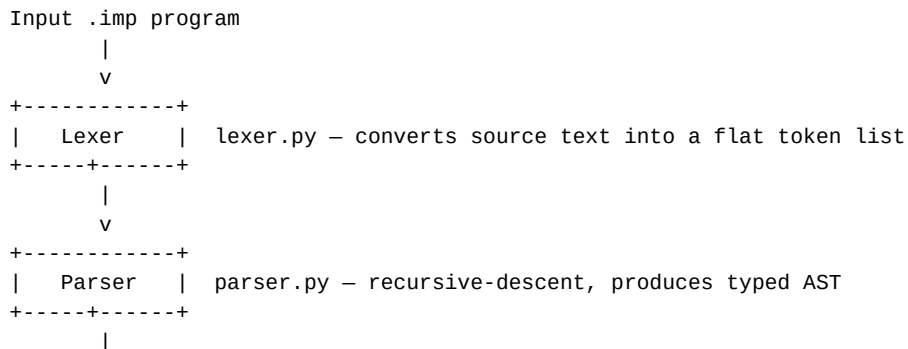
- Variable assignment: `x := expr`
- PII source calls: `get_email()`, `get_name()`, `get_ssn()`, etc.
- String concatenation: `expr + expr`
- Sink statements: `log(expr)`, `send_analytics(expr)`, `send_to_api(expr)`
- Single-line comments: `# ...`

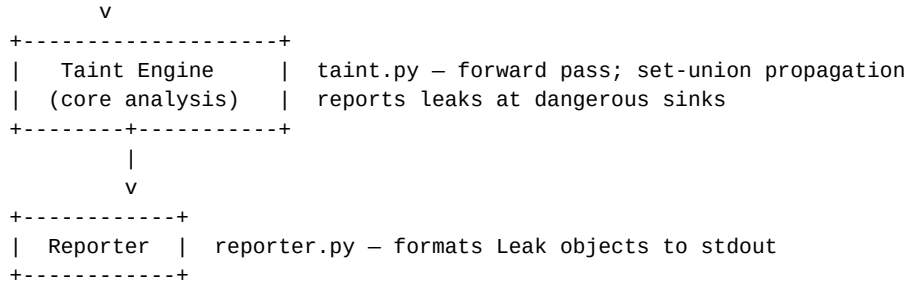
The grammar deliberately excludes conditionals, loops, and user-defined functions to keep the analysis tractable and the implementation focused on the core taint-propagation concept.

2. Architecture & Design

2.1 System Architecture

The tool is a four-stage pipeline:





2.2 Key Data Structures

Structure	Location	Description
Token	lexer.py	Dataclass holding type, value, and line number for each token.
AST nodes (Program, Assignment, SinkCall, BinaryOp, Call, Variable, StringLiteral)	ast_nodes.py	Typed dataclasses representing every grammar construct. Every node stores its source code.
TaintEnv	taint.py	Dict[str, FrozenSet[str]] — maps each variable name to the set of PII tags (e.g. {"email", "password"}).
PII_SOURCES	taint.py	Dict mapping source function names (e.g. get_email) to the frozenset of PII tags they return.
SINK_RECOMMENDATIONS	taint.py	Dict mapping sink function names to human-readable remediation strings.
Leak	taint.py	Dataclass: line number, sink name, frozenset of PII types, recommendation string.

2.3 Core Algorithm

The taint engine performs a single sequential forward pass over the statement list. The environment starts empty (all variables untainted). No fixed-point iteration is needed because the language has no loops or branches — the analysis is linear in the number of statements [4].

```

env = {} # variable -> frozenset of PII tags

for stmt in program.statements:
    if Assignment(target, value):
        env[target] = eval_taint(value, env)

    elif SinkCall(func, args) and is_sink(func):
        combined = union of eval_taint(arg, env) for each arg
        if combined != empty:
            emit Leak(line, func, combined, recommendation(func))

def eval_taint(expr, env):
    Variable x      -> env.get(x, empty)
    StringLiteral  -> empty
    SourceCall f() -> PII_SOURCES[f]
    OtherCall f(a) -> union of eval_taint(a) for each arg # conservative
    BinaryOp a + b -> eval_taint(a) union eval_taint(b)

```

Design decisions: (1) FrozenSets are used so tag sets are hashable and immutable, preventing accidental mutation during propagation. (2) Unknown function calls conservatively propagate the union of their argument taints — this is sound but may introduce false positives when a helper genuinely sanitizes its input. (3) Sinks are identified both by an exact-match table and by substring matching (any

name containing 'log', 'analytics', 'api', etc.) so the tool catches common naming patterns without requiring a complete manifest.

3. Example Runs

Example 1: Spec Case — Two PII Types, Two Sinks (basic.imp)

Demonstrates the canonical case: email and name are merged then sent to a logger and an analytics endpoint.

Input program:

```
email := get_email()
name  := get_name()
user_info := email + " - " + name
log(user_info)          # Privacy leak!
send_analytics(name)    # Another leak!
```

Tool output:

```
PII LEAK at line 4
Sink: log
PII types exposed: {email, name}
Recommendation: Redact PII before logging

PII LEAK at line 5
Sink: send_analytics
PII types exposed: {name}
Recommendation: Anonymize before sending to analytics
```

Explanation: At line 3, `user_info` receives the union of the email and name tags. At line 4, both tags are exposed at the log sink. At line 5, only the name tag flows through. Both are correctly flagged with sink-specific recommendations.

Example 2: Multiple PII Types Across Three Sinks (multi_sink.imp)

Four PII types assigned separately; a clean string is also logged to confirm no false positive.

Input program:

```
ssn      := get_ssn()
phone    := get_phone()
address  := get_address()
name     := get_name()
record   := ssn + " " + name
greeting := "Hello, welcome!"
log(record)          # leaks ssn + name
send_analytics(phone) # leaks phone
send_to_api(address)  # leaks address
log(greeting)        # clean — no leak
```

Tool output:

```
PII LEAK at line 13
Sink: log
PII types exposed: {name, ssn}
```

Recommendation: Redact PII before logging

PII LEAK at line 14

Sink: send_analytics

PII types exposed: {phone}

Recommendation: Anonymize before sending to analytics

PII LEAK at line 15

Sink: send_to_api

PII types exposed: {address}

Recommendation: Remove PII before sending to external sink

Explanation: Three distinct leaks are correctly reported across three sink types. The final `log(greeting)` produces no report because `greeting` is a plain string literal with an empty tag set — confirming the tool does not over-report.

Example 3: No Leak — PII Assigned but Never Reaches a Sink (no_leak.imp)

Input program:

```
name := get_name()
greeting := "Hello, user!"
log(greeting)
```

Tool output:

No PII leaks detected.

Explanation: Although `name` is tainted, it is never passed to any sink. Only the clean string `greeting` reaches `log`. The tool correctly reports no leaks.

Example 4: Chained Propagation — PII Through Three Variable Hops (chained.imp)

Verifies that the tag set is preserved through multiple intermediate assignments.

Input program:

```
email := get_email()
a := email
b := a
c := b + " extra"
log(c)
```

Tool output:

PII LEAK at line 6

Sink: log

PII types exposed: {email}

Recommendation: Redact PII before logging

Explanation: The email tag propagates through `a` → `b` → `c` unchanged. Concatenation with a plain string literal does not dilute the tag set. The leak is correctly detected three assignments removed from the original source.

4. Individual Contributions

Member 1: Justin Veltri

Category	Contribution
Role	Project Lead & Taint Engine
Modules Owned	taint.py, main.py
Key Tasks	1. Designed overall system architecture and module interfaces 2. Implemented the taint propagation engine (taint.py) 3. Defined PII source and sink tables with substring fallback matching 4. Implemented CLI entry point (main.py) 5. Led integration, debugging, and final system verification
Estimated Effort	20%
Self-Assessment	Designing the taint engine was straightforward once the data structures were settled. The main challenge

Member 2: Flavia Daniels

Category	Contribution
Role	Parser & AST Design
Modules Owned	parser.py, ast_nodes.py
Key Tasks	1. Designed all AST node dataclasses (ast_nodes.py) 2. Implemented the recursive-descent parser (parser.py) 3. Handled one-token lookahead to distinguish assignments from sink calls 4. Threaded source line numbers through every AST node 5. Validated parser output against all supported grammar constructs
Estimated Effort	20%
Self-Assessment	The trickiest part was disambiguating statements that start with an identifier — both assignments and sin

Member 3: Arnav Vasa

Category	Contribution
Role	Documentation
Modules Owned	README.md, project report
Key Tasks	1. Wrote all sections of the project report 2. Wrote the README with build, run, and test instructions 3. Documented the PII source and sink tables 4. Proofread and formatted all written deliverables
Estimated Effort	20%
Self-Assessment	Translating technical implementation details into clear written explanations required close collaboration v

Member 4: Garrett Boag

Category	Contribution
Role	Lexer & Test Suite
Modules Owned	lexer.py, tests/
Key Tasks	1. Implemented the regex-based tokenizer (lexer.py) 2. Wrote 6 lexer unit tests (test_lexer.py) 3. Wrote 7 parser unit tests (test_parser.py) 4. Wrote 10 taint analysis unit tests (test_taint.py) 5. Verified all 23 tests pass on a clean environment
Estimated Effort	20%
Self-Assessment	Writing the test suite forced me to think carefully about edge cases like comments, chained assignments

Member 5: Jonathan Veltri

Category	Contribution
Role	Integration, Reporter & Examples
Modules Owned	reporter.py, examples/
Key Tasks	1. Implemented the output formatter (reporter.py) 2. Created all four example .imp programs (examples/) 3. Ran end-to-end integration tests across all modules 4. Verified tool output matches the project spec exactly 5. Coordinated the final submission checklist
Estimated Effort	20%
Self-Assessment	Matching the exact output format specified in the project description required several iterations. End-to-e

Contribution Summary

Member	Design	Implementation	Testing	Documentation	Presentation	Overall %
Justin Veltri	High	High	Medium	Low	Medium	20%
Flavia Daniels	High	High	Medium	Low	Medium	20%
Arnav Vasa	Low	Low	Low	High	Medium	20%
Garrett Boag	Low	High	High	Low	Medium	20%
Jonathan Veltri	Medium	Medium	Medium	Medium	High	20%

Note: Effort percentages sum to 100%.

5. Challenges & Lessons Learned

5.1 Technical Challenges

Challenge 1 — Grammar ambiguity between assignments and sink calls. Both statement forms begin with an identifier, so the parser must inspect the second token to distinguish `x := ...` from `f(...)`. Implementing clean one-token lookahead in a recursive-descent parser without backtracking required careful sequencing of the consume/peek calls and explicit error messages for unexpected token types.

Challenge 2 — Defining a sound but practical sink boundary. Hardcoding an exact list of sink names is fragile; real codebases have many logging helpers with inconsistent names. We addressed this with a two-tier approach: an exact-match table for the most common sinks, plus substring matching on the function name for everything else. This reduced missed leaks without requiring a full configurable manifest.

5.2 Lessons Learned

- Set-based taint tracking is surprisingly expressive for a first-order language — the core engine is under 60 lines of Python and handles all required cases.
- Line number tracking must be wired through every AST node from the lexer onward. Retrofitting it after the parser is written is significantly more painful.
- Separating concerns cleanly (lexer → parser → taint → reporter) made unit testing each layer in isolation straightforward and kept the overall test suite small and focused.
- Static analysis tools benefit from explicit, human-readable output — the recommendation strings per sink type make the tool actionable, not just informational.

6. Limitations & Future Work

6.1 Known Limitations

Limitation	Impact
No control flow (branches, loops)	Cannot track PII on different execution paths — may miss leaks in conditional branches or loops
No sanitization modeling	A function that redacts or anonymizes PII is treated conservatively, potentially producing false positives
Intraprocedural only	PII passed into or returned from user-defined functions is not tracked across call boundaries
Fixed source/sink lists	Functions not in the built-in list or whose names do not match the substring heuristic will be missed
No object or field tracking	PII stored in a struct field (e.g. <code>user.email</code>) is not tracked; only top-level variables are modeled

6.2 Potential Extensions

- **Control-flow sensitivity:** extend to `if/else` and `while` so PII on different branches is tracked independently, reducing false positives.
- **Sanitization modeling:** introduce a `redact()` primitive that clears all PII tags, modelling real anonymization operations [2][3].
- **Interprocedural analysis:** track PII across user-defined function boundaries using a call graph and summary-based propagation [4][5].

- **Configurable manifests:** allow teams to supply a JSON file of project-specific source and sink function names, removing the dependency on the built-in lists.
- **IDE integration:** surface leak warnings as inline annotations in VS Code or IntelliJ to catch issues at the point of authorship.

7. References

- [1] W. Enck, P. Gilbert, B. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones," in *Proc. USENIX OSDI*, 2010.
- [2] European Parliament, "General Data Protection Regulation (GDPR), Recital 26: Not Applicable to Anonymous Data," *Official Journal of the European Union*, 2016.
- [3] National Institute of Standards and Technology, "Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0," NIST, Jan. 2020. [Online]. Available: <https://www.nist.gov/privacy-framework>. Accessed: Apr. 27, 2026.
- [4] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Pearson, 2006. (Ch. 9: Machine-Independent Optimizations — dataflow analysis foundations.)
- [5] F. Tip, "A Survey of Program Slicing Techniques," *Journal of Programming Languages*, vol. 3, no. 3, pp. 121–189, 1995.